Institute for Complex Systems

JOHANNES KEPLER UNIVERSITÄT LINZ

Univ.-Prof. Dr. Daniel Große
DI Simon Reitinger & Katharina Ruep, M.Sc.

30. April 2026

Übung Projekt
zur Vorlesung Rechnerarchitektur

Abgabe bis **Donnerstag 28 Mai, 2026 10:15** via EPICS: `https://ep.ics.jku.at`.

Im Rahmen des Projekts soll der in der Vorlesung vorgestellte RISC-V Single-Cycle Prozessor erweitert und in VHDL realisiert werden.
Für das Projekt steht in Moodle ein Template des Prozessors zur Verfügung, d.h. der Prozessor setzt bereits die in der Vorlesung vorgestellten Instruktionen um. Die erweiterten VHDL Dateien müssen vor der Abgabe in EP.ICS mit *ghdl* ausgeführt werden. Dateien, die nicht kompilieren, können nicht abgegeben werden.

Aufgabe 1 – Konzeptionelle Erweiterung des Prozessors (14 Punkte)

Der in der Vorlesung vorgestellte Single-Cycle RISC-V Prozessor soll erweitert werden, damit er einen Algorithmus zum Summieren eines Byte Arrays unterstützt. Hierzu müssen die Instruktionen **jalr**, **auipc** und **lbu** implementiert werden. Erweitere dazu in Abbildung 2 für jeden dieser Befehle (falls nötig) den Datenpfad und füge weitere Kontrollsignale und Register hinzu.[1] Gib für jeden dieser Befehle auch den Wert aller Kontrollsignale an. Nimm dabei an, dass die ALU aus Abbildung 1 verwendet wird und gib `don't cares` explizit mit X an.
Die Abgabe soll ein Bild mit dem erweitertem Daten- und Kontrollpfad des Prozessors pro zusätzlichem Befehl (es können auch mehrere Erweiterungen in ein Bild gezeichnet werden) sowie eine Tabelle mit den Werten aller Kontrollsignale für die neuen Befehle beinhalten. Hinweis: Entnimm die Semantik der einzelnen RISC-V Befehle der in Moodle zur Verfügung gestellten RISC-V Reference Card[2].

Aufgabe 2 – Implementierung in VHDL (24 Punkte)

Implementiere den erweiterten Single-Cycle RISC-V Prozessor in VHDL. Verwende dazu die im Template vorgegebenen Strukturen und füge (wenn nötig) weitere Komponenten hinzu. Durch das Erweitern des Prozessors kann es sein, dass sich die Schnittstelle einiger Komponenten ändert.

---

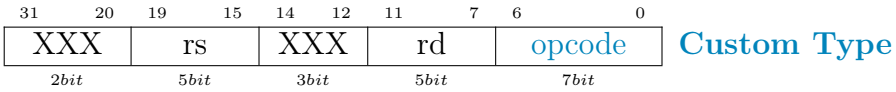
[1] Der in schwarz gezeichnete Teil in Abbildung 1 und Abbildung 2 visualisiert den Datenpfad, während der Kontrollpfad in Blau gezeichnet ist.

[2] `https://moodle.jku.at/mod/resource/view.php?id=12898432`

Um den erweiterten Prozessor zu testen, gibt es ein Beispielprogramm in `code/sum_bytes.s`. Das Programm ladet das Array `[10,240,120,200,180,50,170,210]` und summiert es Byte für Byte.
Um das Programm mit deinem Prozessor auszuführen, verwende den schon kompilierten Maschinencode in `files/text.txt` mit den Daten in `files/data.txt`. Nutze dafür die *Makefile*, welche das Programm und den Datenspeicher mit den jeweiligen Dateien befüllt und so den Prozessor auf die Ausführung vorbereitet. Eine Testbench zum Simulieren ist auch im Template enthalten, welche den Prozessor treibt und Print/Exit ECalls behandelt. Verwende `make`, um den Prozessor zu starten!
Am Schluss wird der Wert ausgegeben. Kontrolliere, ob die richtige Zahl 1180 ausgegeben wird.
Hinweis: Bei der Simulation in EP.ICS müssen auch die .txt Programmdateien ausgewählt werden. Zusätzlich kann es Sinn machen, eigene kleine Testprogramme zu schreiben. Diese können mit der "Dump" Funktion im Venus Simulator in eine hexadezimale Darstellung übersetzt werden, die durch den Single-Cycle Prozessor ausgeführt werden können.

Aufgabe 3 – Integration einer Custom Instruction (10 Punkte)

Die Implementation in der vorherigen Aufgabe verwendet eine Addition pro Byte im Array. Nun soll dieses Addieren der Bytes durch das Hinzufügen der *Custom Instruction* `bytesum` beschleunigt werden. Die Instruktion hat folgendes Format:



`bytesum` lest ein 32 Bit Registerwort, teilt es in 4 Bytes (unsigned) und speichert deren Summe in einem Register ab:

$$rd = \text{ZeroExt}(rs_{[7:0]}) + \text{ZeroExt}(rs_{[15:8]}) + \text{ZeroExt}(rs_{[23:16]}) + \text{ZeroExt}(rs_{[31:24]})$$

Der *opcode* der Instruktion ist `0001011`. "XXX"beschreibt die *don't care* Teile der Instruktion und sollen beim Dekodieren ignoriert werden.
Zum Testen der Instruktion kann der Programmcode in `text_custom_instruction.txt` verwendet werden. Dazu direkt in der Makefile den Pfad ändern oder
`make TEXT_FILE=files/text_custom_instruction.txt` ausführen. Das Programm verwendet die neue Instruktion und sollte dasselbe Ergebnis (1180) wie in Aufgabe 2 liefern.
Hinweis: Für die Simulation in EP.ICS muss `files/text_custom_instruction.txt` als `text.txt` abgegeben werden.

Aufgabe 4 – (Bonus) BLT Instruktion (2 Punkte)

Finde heraus, wie die `blt` Instruktion für den Prozessor `ICS-NEX-RV32I-SC` funktioniert. Verwende dafür SonicRV (`https://sonic-rv.ics.jku.at/simulate`), wähle den Prozessor aus und schreibe ein Assembly Programm zum Ausprobieren der `blt` Instruktion. Notiere dann den Ablauf der Instruktion, i.e. wie die Kontrollsignale gesetzt werden, welche Werte berechnet werden und welche Komponenten dafür notwendig sind.

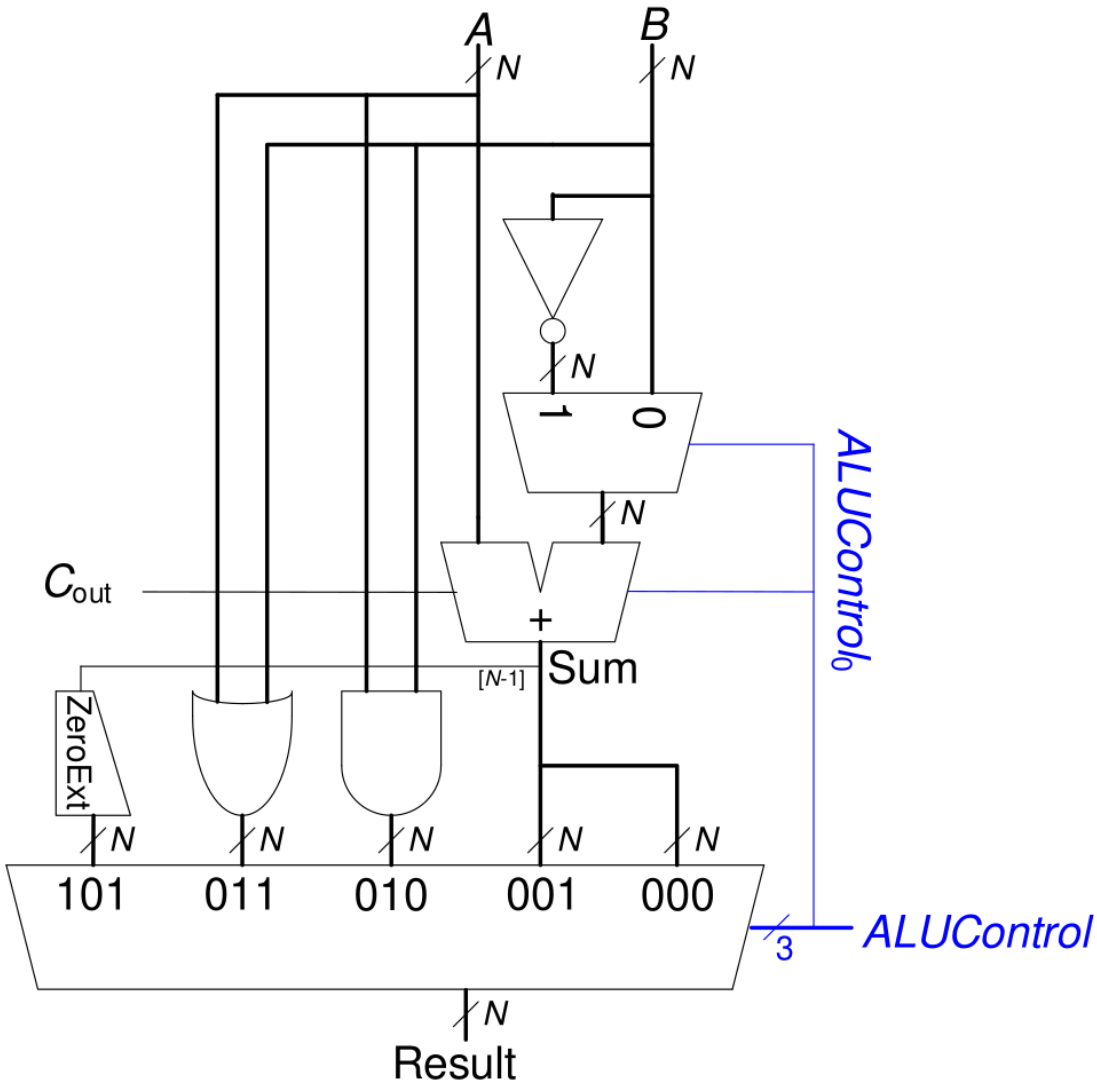


Abbildung 1: ALU des Single-Cycle RISC-V Prozessors

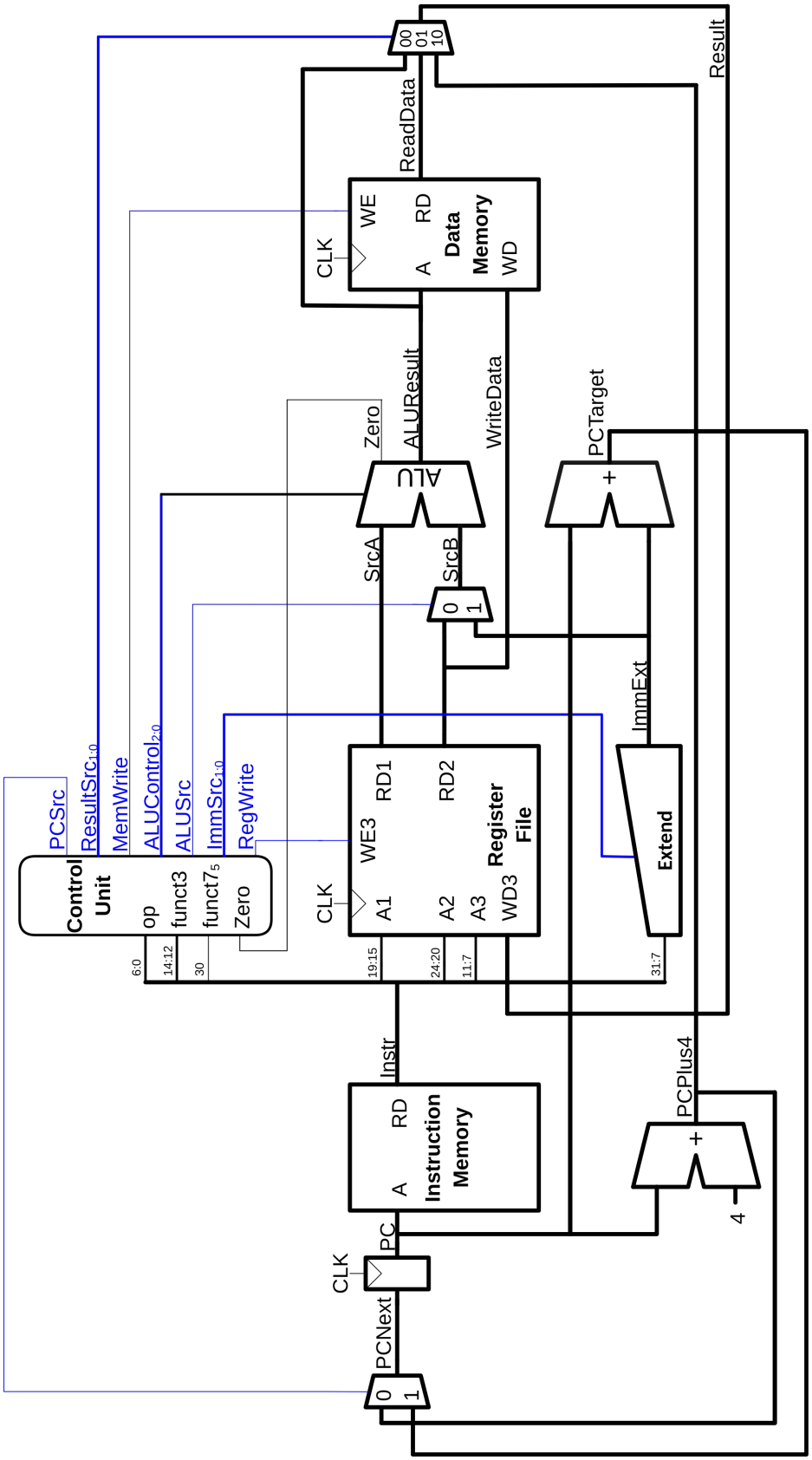


Abbildung 2: Schaltbild des Single-Cycle RISC-V Prozessors